

CASSA Observatory

Master Documentation

Hardware Characterization & Data Pipeline Architecture

Part I: Astronomical CMOS Sensor Characterization SOP

Part II: Pipeline Installation & Deployment (`cassa-photometry`)

Part III: Image Processing Pipeline Architecture (Phases 1–4)

Part IV: Atmospheric Seeing Monitor (`cassa-dimm`)

August 27, 2026

Contents

Part I: Sensor Characterization SOP	2
1 Astronomical CMOS Sensor Characterization	2
1.1 Prerequisites & Lab Setup	2
1.1.1 Hardware Requirements	2
1.1.2 Software Configuration	2
1.2 Phase 1: Statistical Characterization (Photon Transfer Curve Method)	2
1.2.1 Readout Noise (σ_R)	2
1.2.2 System Gain (g) & Full Well Capacity (FWC)	3
1.2.3 Dynamic Range (DR)	4
1.3 Phase 2: Thermal Characterization	5
1.3.1 Dark Current (I_D)	5
1.3.2 Dark Current Linearity (Amp Glow Check)	5
1.4 Phase 3: Optical Characterization	5
1.4.1 Absolute Quantum Efficiency (QE)	6
1.4.2 Filter Transmission Curves (Multi-Filter Profiling)	6
1.4.3 Summary Reference Table of Characterization Parameters	7
Part II: Pipeline Environment Deployment	8
2 Installation & Deployment Guide	8
2.1 Install Miniconda3	8
2.2 Prepare the Pipeline Configuration	8
2.3 HPC Network Drive Bypass (Critical for Servers)	9
2.4 Build the Environment	9
2.5 Install the <code>cassa-photometry</code> Package	10
2.6 Astrometry Index Files (Required for Phase 2)	10
2.7 Verification	11
2.8 Pipeline Execution Guide	11
2.8.1 Running Phase 1 (Instrument Signature Removal)	11
2.8.2 Running Phase 2 (Integration & Astrometry)	11
2.8.3 Running Phase 3 (Photometry & Zero Point)	12
2.8.4 Running Phase 4 (Diagnostics)	12
2.8.5 End-to-End and Verification	12
2.8.6 Configuration Overrides	12
Part III: Image Processing Architecture	13
3 Pipeline Overview & Data Model	13
3.1 The Three-Plane Data Model (SCI / ERR / DQ)	13
4 Data Flow Architecture: Phase 1 (ISR)	14

5	Phase 1: Instrument Signature Removal (ISR)	15
5.1	Master Calibration Processes	15
5.1.1	Master Bias Generation (Median Combine)	15
5.1.2	Master Dark Generation (Subtract Bias & Median Combine)	15
5.1.3	Master Flat Generation (Subtract, Normalize, Median Combine)	15
5.2	The Science Processing Flow	16
6	Data Flow Architecture: Phase 2 (Integration & Astrometry)	17
7	Phase 2: Integration, Astrometry & QA Engine	18
7.1	Grouping and Telemetry Engine	18
7.1.1	Metadata Skim (Target Grouping)	18
7.1.2	Telemetry (Anchor Selection)	18
7.2	Integration Engine	18
7.2.1	Geometric Registration & Flux Scaling	18
7.2.2	Stacking (Inverse Variance Weighting & 3-Sigma Clipping)	18
7.3	Astrometry Engine	19
7.3.1	Plate Solving & Rescue Pass	19
8	Data Flow Architecture: Phase 3 (Photometry & Zero Point)	20
9	Phase 3: Photometry, Zero Point & Catalogs	21
9.1	Reference-Catalog Cross-Match	21
9.1.1	Priority Cascade	21
9.2	Filter-Wise Zero Point	21
9.2.1	Differential Photometry with Uncertainty	21
9.3	Products	21
9.3.1	Flux Calibration & Catalog	21
10	Data Flow Architecture: Phase 4 (Diagnostics)	22
11	Phase 4: Diagnostics & Validation	23
11.1	PSF & Image Quality	23
11.2	Per-Stage Checks	23
11.3	Outputs	23
12	Appendix: Technical Implementation Details	24
12.1	Dynamic FITS Header Extraction (The Adapter Pattern)	24
12.2	The Mathematics of Flat Frame Normalization	24
12.3	Overscan Banding Correction	24
12.4	Laplacian Cosmic Ray Detection (<code>astroscrappy</code>)	24
12.5	Bad Pixel Mask Generation (BPM)	25
12.6	Strict Dark Current Matching vs. Scaling	25
12.7	Sub-Pixel Interpolation Physics	25
12.8	Robust Noise Estimation (Median Absolute Deviation)	26
12.9	Asterism Geometric Hashing (<code>Astroalign</code>)	26
12.10	Integration Mathematics (Weighting & Clipping)	26
12.11	Blind Plate Solving Optimization	26
12.12	Seeding the Error Budget (<code>create_deviation</code>)	26
12.13	The Data-Quality Bitmask (DQ)	27
12.14	Filter-Wise Zero Point with Uncertainty	27
12.15	PSF Estimation via 2D Gaussian Fitting	27

Part IV: Atmospheric Seeing Monitor (cassa-dimm)	28
13 DIMM Overview & Deployment	28
13.1 Principle of Operation	28
13.2 Installation	28
13.3 Running the Monitor	28
13.4 Trying It with Simulated Data	29
14 Data Flow Architecture: DIMM Seeing Monitor	30
15 DIMM Reduction Engine	31
15.1 Multi-Star Detection & Doublet Pairing	31
15.2 Sub-Pixel Centroiding	31
15.3 Seeing Physics	31
15.4 Airmass Correction & Multi-Star Combination	31
15.5 Continuous Monitoring & Quality Control	32
15.6 Wide-Field Operation (0.5° Fields)	32
16 Appendix: DIMM Technical Details	32
16.1 Differential Image Motion & the Fried Parameter	32
16.2 Two-Pass Centroiding Against Tracking Motion	33

Part I: Sensor Characterization SOP

1 Astronomical CMOS Sensor Characterization

This document serves as a comprehensive manual for the physical and statistical characterization of astronomical imaging sensors (such as the QHYCCD miniCAM8M / IMX585). It outlines the precise methodologies, required equipment, FITS metadata configuration, and graphing techniques used to extract core operational parameters: System Gain, Readout Noise, Full Well Capacity, Dynamic Range, Dark Current, Absolute Quantum Efficiency (QE), and Filter Transmission Curves.

1.1 Prerequisites & Lab Setup

1.1.1 Hardware Requirements

- **Device Under Test (DUT):** Cooled CMOS camera (e.g., QHYCCD miniCAM8M containing the Sony IMX585 sensor).
- **Power & Connectivity:** 12V DC power supply for the Thermoelectric Cooler (TEC) and a high-quality USB 3.0 data cable.
- **Light-Tight Seal:** A completely opaque, threaded metal lens cap or a dedicated laboratory dark box.
- **Light Source:** A highly stable, dimmable, non-flickering LED flat-field panel or uniform integrating sphere.

1.1.2 Software Configuration

- **Capture Interface:** Software capable of saving raw images natively in `.fit` or `.fits` format.
- **Data Structure:** The automated processing script relies on strict directory routing rather than explicit file names. Create a parent workspace directory with mandatory subfolders: `../data/bias/`, `../data/dark/`, `../data/flat/`, and `../data/spectra/`.

Critical Verification Step: Before initiating a large-scale capture campaign, take a single test exposure. Open the FITS file programmatically or via a viewer (e.g., DS9) to confirm your capture software writes the specific metadata keys as `IMAGETYP`, `GAIN`, `EXPTIME`, and `CCD-TEMP`. If your software uses alternate keys (e.g., `EGAIN`, `SET-TEMP`), record them so they can be re-mapped in the analysis script.

1.2 Phase 1: Statistical Characterization (Photon Transfer Curve Method)

The Photon Transfer Curve (PTC) method utilizes the statistical nature of light (Poisson distribution) to measure sensor characteristics without requiring an absolute calibrated photon flux.

1.2.1 Readout Noise (σ_R)

Readout noise is the baseline electronic noise introduced by the sensor's amplifiers and analog-to-digital converters (ADC) when reading the pixel array, measured in electrons (e^-).

- **Setup:** Seal the camera completely using the opaque lens cap. Ensure the room is dark to eliminate potential light leaks.
- **Thermal Control:** Turn on the TEC cooler and set it to a stable baseline standard operating temperature (e.g., -10.0°C). Wait 5 minutes to achieve thermal equilibrium.
- **Target Sweep Settings:** Prepare to cycle through standard driver gain steps (e.g., 0, 50, 100, 150, 200).
- **FITS Header Requirements:** For every frame captured in this step, ensure the software writes: `IMAGETYP: 'Bias' or 'Bias Frame'`, `EXPTIME: 0.000011` (or the absolute minimum hardware limit of the camera), `CCD-TEMP: -10.0` (Must remain static), `GAIN: [Current test setting, e.g., 0, 50, 100, 150, 200]`.
- **Capture Sequence:** Set the camera to the first gain setting, verify the header parameters, and capture a sequence of 5 to 10 identical frames. Step to the next gain setting, verify the header updates, and repeat.
- **Storage:** Save all files directly into the `/bias` folder.
- **Mathematical Analysis:**

1. Subtract Bias Frame 2 from Bias Frame 1 pixel-by-pixel to isolate temporal noise from fixed spatial structure:

$$\Delta I_{bias} = I_{bias1} - I_{bias2}$$

2. Compute the standard deviation of the difference frame (σ_{diff}).
3. Adjust for the mathematical combination of two frames and multiply by the system gain (g , estimated from flat frames; see System Gain section) to convert from ADU to electrons:

$$\sigma_R = \left(\frac{\sigma_{diff}}{\sqrt{2}} \right) \times g$$

- **Graphing:** Plot Readout Noise (e^-) on the Y-axis against Camera Driver Gain Setting on the X-axis.

1.2.2 System Gain (g) & Full Well Capacity (FWC)

System gain is the conversion factor transforming physical electrons into digital counts (ADU). Full Well Capacity represents the maximum electron threshold a pixel can hold before saturation occurs.

- **Setup:** Remove the lens cap. Place the stable flat-field light source flush against the camera opening.
- **Thermal Control:** Maintain the TEC cooler at your standard operating baseline temperature (e.g., -10.0°C).
- **Boundary Calibration:** Before recording, set the camera to Gain 0. Adjust the physical brightness of your LED panel so that a 0.1-second exposure is nearly pitch black (just above the bias pedestal) and a 10.0-second exposure completely saturates the sensor at 16-bit limits (65,535 ADU). Do not alter the light panel brightness after establishing these bounds.
- **FITS Header Requirements:** For every frame captured in this step, ensure the software writes: `IMAGETYP: 'Flat' or 'Flat Frame'`, `CCD-TEMP: -10.0` (Must remain static), `GAIN: [Current test setting, e.g., 0, 50, 100, 150, 200]`, `EXPTIME: [The specific exposure step duration in seconds]`.

- **The Exposure Sweep Sequence:** Set the camera to the first target gain (e.g., Gain 0). Set the exposure time to 0.1s. Capture exactly TWO identical frames. Step up the exposure duration incrementally (e.g., 0.5s, 1.0s, 1.5s ... up to 10.0s). At each step, capture exactly TWO identical frames. Ensure the EXPTIME updates correctly in the header metadata. Stop stepping when the frames reach full ADU saturation.
- **The Gain Sweep Sequence:** Change the camera to the next target gain (e.g., Gain 50) and repeat the entire exposure sweep from 0.1s up to saturation in pairs. Repeat this process for all target gains.
- **Storage:** Save all files directly into the /flat folder.
- **Mathematical Analysis:**

1. For each pair of flats at a given exposure and gain, calculate the global mean signal:

$$\bar{I} = \frac{\text{mean}(I_{FlatA}) + \text{mean}(I_{FlatB})}{2}$$

2. Subtract the two frames pixel-by-pixel and calculate the variance of the difference to eliminate spatial flat-field variations:

$$\sigma_{diff}^2 = \text{var}(I_{FlatA} - I_{FlatB}) \rightarrow \sigma_{signal}^2 = \frac{\sigma_{diff}^2}{2}$$

3. Isolate the linear region of the data (typically 10% to 70% of maximum saturation). Fit a first-order polynomial mapping Variance (σ_{signal}^2) as a function of Mean ($\bar{I}$). The system gain is the inverse of the linear slope:

$$g = \frac{1}{\text{slope}}$$

4. Locate the saturation index where variance peaks and abruptly drops. Multiply the Mean ADU value directly preceding this drop by the system gain to determine the Full Well Capacity:

$$\text{FWC} (e^-) = \bar{I}_{saturation} \times g$$

- **Graphing:** PTC Graph: Plot Signal Variance (ADU^2) on the Y-axis against Mean Signal (ADU) on the X-axis using a log-log scale. FWC Graph: Plot Full Well Capacity (e^-) on the Y-axis against Camera Driver Gain Setting on the X-axis.

1.2.3 Dynamic Range (DR)

Dynamic range mathematically defines the ratio between the largest non-saturating signal and the quietest detectable signal in a single exposure.

- **Analysis:** Utilize the computed FWC and Readout Noise values derived at each specific driver gain setting.
- **Equations:**

$$\text{Dynamic Range (Stops)} = \log_2 \left(\frac{\text{Full Well Capacity} (e^-)}{\text{Readout Noise} (e^-)} \right)$$

- **Graphing:** Plot Dynamic Range (Stops) on the Y-axis against Camera Driver Gain Setting on the X-axis.

1.3 Phase 2: Thermal Characterization

This phase measures dark current—the rate at which ambient thermal energy excites electrons within the silicon substrate, mimicking physical photon arrivals.

1.3.1 Dark Current (I_D)

- **Setup:** Reseal the camera securely with the opaque lens cap.
- **Static Sensor Settings:** Set the camera to a fixed gain configuration (typically baseline Gain 0) and a long, fixed exposure time (e.g., 300.0 seconds).
- **FITS Header Requirements:** For every dark frame captured, ensure the software writes: `IMAGETYP: 'Dark' or 'Dark Frame'`, `GAIN: 0` (Must remain static), `EXPTIME: 300.0` (Must remain static), `CCD-TEMP:` [The specific stabilized temperature step in Celsius].
- **The Temperature Sweep Sequence:** Set the TEC cooler to +20.0°C. Wait 3 to 5 minutes for the internal temperature sensor to settle perfectly. Verify the header matches, then record a single 300-second dark exposure. Lower the TEC temperature in 5°C decrements (e.g., +15°C, +10°C, +5°C ... down to -20°C). At each step, allow the temperature to stabilize completely, verify the header metadata updates, and capture a 300-second exposure.
- **Storage:** Save all files directly into the `/dark` folder.
- **Mathematical Analysis:** Isolate the thermal signal component by subtracting the average baseline value of a bias frame ($\bar{I}_{bias}$) from the mean dark frame signal ($\bar{I}_{dark}$). Convert the thermal signal from digital counts into physical electrons using the calculated Gain, then normalize over time:

$$\bar{I}_{thermal} = \bar{I}_{dark} - \bar{I}_{bias} \quad \rightarrow \quad I_D = \frac{\bar{I}_{thermal} \times g}{EXPTIME}$$

- **Graphing:** Plot Dark Current ($e^-/\text{pixel/s}$) on a logarithmic Y-axis against Sensor Temperature (°C) on a linear X-axis to display exponential thermal decay.

1.3.2 Dark Current Linearity (Amp Glow Check)

Verifies thermal buildup is perfectly linear over time and checks for localized sensor heating.

- **Setup:** Camera sealed. Gain 0. TEC locked to 0.0°C.
- **Exposure Sweep Sequence:** Capture dark frames at progressively longer exposures (e.g., 10s, 30s, 60s, 120s, 300s, 600s).
- **Mathematical Analysis:**

$$\text{Total Thermal Electrons} = \bar{I}_{thermal} \times g$$

- **Graphing:** Plot Total Thermal Signal (e^-) vs. Exposure Time (s). Fit an ideal linear trendline to visually check for exponential curvature indicating Amp Glow.

1.4 Phase 3: Optical Characterization

This phase maps the precise probability that an incoming photon of a specific wavelength will be converted into a measurable electron.

1.4.1 Absolute Quantum Efficiency (QE)

Pipeline Script Behavior Note: Calculating a physical QE curve requires highly specialized laboratory equipment to measure absolute photon fluxes. Because standard calibration frames (Bias, Dark, Flat) do not contain wavelength information, the default diagnostic script contains a hardcoded mathematical representation of the Sony IMX585 mono sensor curve to prevent software crashes and complete the 7-panel dashboard layout. For empirical laboratory testing, swap the simulated data stream with a structured `.csv` file compiled from physical instruments.

- **Laboratory Setup:** Align a regulated Quartz Tungsten Halogen (QTH) lamp, a monochromator, and an integrating sphere on an optical bench in a light-tight chamber.
- **Reference Calibration:** Place a calibrated reference photodiode at the exit port of the integrating sphere. Cycle the monochromator from 350 nm to 1000 nm in 5 nm steps. Record the photodiode output current in Amperes, and convert it into absolute photon flux (photons/second/cm²) using its certified calibration table.
- **Sensor Characterization:** Remove the photodiode and mount the camera sensor at the exact same spatial focal plane of the exit port. Step the monochromator through the identical 350 nm to 1000 nm sequence. Take an exposure at each step, and map the results.
- **Calculation:** Convert the digital signal from ADU to total electrons using your established System Gain. Calculate the percentage efficiency at each discrete wavelength:

$$\text{QE (\%)} = \left(\frac{\text{Electron Generation Rate}}{\text{Absolute Photon Flux Rate}} \right) \times 100$$

- **Graphing:** Plot Absolute QE (%) on the Y-axis against Wavelength (nm) on the X-axis, typically filling the region underneath the curve for standard spec sheet reporting.

1.4.2 Filter Transmission Curves (Multi-Filter Profiling)

Measures the specific wavelength bandpass efficiency of inserted optical filters (e.g., H-Alpha, OIII, LRGB).

- **Setup:** Utilize the same optical bench as the QE test (QTH Lamp + Monochromator + Integrating Sphere).
- **Calibration (Unfiltered Baseline):** Place the calibrated reference photodiode (or the characterized CMOS sensor) at the exit port. Sweep the target wavelength range (e.g., 350nm to 1000nm) and record the baseline signal intensity ($I_{unfiltered}$) at each step.
- **Filter Measurement:** Insert the first target astronomical filter directly between the integrating sphere exit port and the sensor. Run the exact same wavelength sweep, recording the new signal intensity ($I_{filtered}$) at each step. **Repeat this entire step for every filter in the wheel** (e.g., Luminance, Red, Green, Blue, H-Alpha, OIII, SII).
- **Calculation:** Calculate the transmission percentage for each wavelength across every individual filter:

$$\text{Transmission (\%)} = \left(\frac{I_{filtered}}{I_{unfiltered}} \right) \times 100$$

- **Graphing:** Overlay the Transmission (%) of all tested filters onto a single, color-coded graph. Plot Transmission on the Y-axis against Wavelength (nm) on the X-axis to visualize the overlapping bandwidth cutoffs, spectral gaps, and peak throughputs of the complete filter set.

1.4.3 Summary Reference Table of Characterization Parameters

Parameter	Unit	Hardware Required	Mandatory Headers	Standard Output Graph
System Gain (g)	e^-/ADU	Uniform light panel, Cooled Camera	IMAGETYP, GAIN, EXP-TIME, CCD-TEMP	Photon Transfer Curve (Log-Log Variance vs. Mean)
Readout Noise (σ_R)	e^-	Opaque Lens Cap, Cooled Camera	IMAGETYP, GAIN, EXP-TIME, CCD-TEMP	Readout Noise vs. Camera Driver Gain Setting
Full Well Capacity	e^-	Uniform light panel, Cooled Camera	IMAGETYP, GAIN, EXP-TIME, CCD-TEMP	Full Well Capacity vs. Camera Driver Gain Setting
Dynamic Range	Stops	Mathematically derived from FWC and Read Noise	Derived programmatically	Dynamic Range vs. Camera Driver Gain Setting
Dark Current (I_D)	$e^-/\text{px/s}$	Opaque Lens Cap, Dynamic TEC Cooler	IMAGETYP, GAIN, EXP-TIME, CCD-TEMP	Dark Current vs. Sensor Temperature (Semi-Log Y)
Dark Linearity	e^-	Lens Cap, TEC Cooler	IMAGETYP, GAIN, EXP-TIME, CCD-TEMP	Thermal Signal vs. Time
Quantum Efficiency	%	Monochromator, Integrating Sphere, Reference Diode	External calibrated data logging (.csv)	Absolute Quantum Efficiency vs. Wavelength (nm)
Filter Trans.	%	Monochromator, Sphere, Ref. Diode	External calibration log	Filter Transmission vs. Wavelength

Part II: Pipeline Environment Deployment

2 Installation & Deployment Guide

This section covers the full setup of the `cassa-photometry` pipeline from scratch on Linux systems. The workflow is: (1) build a reproducible Conda environment that provides the scientific libraries and the `solve-field` binary, (2) install the `cassa-photometry` package to register its command-line tools, and (3) provide the astrometry index files. Instructions for bypassing HPC network-drive database locks are included, as these are commonly encountered on mounted network storage.

2.1 Install Miniconda3

If Miniconda3 is already installed and initialized, skip to the next step.

1. **Download the latest Miniconda installer** for Linux using `wget`:

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -O
miniconda_installer.sh
```

2. **Run the installer in silent mode.** Change the `-p` path to install it in a specific directory (e.g., a mounted hard drive instead of the default OS drive):

```
bash miniconda_installer.sh -b -p ~/miniconda3
```

3. **Initialize Conda** so it integrates with your shell profile:

```
~/miniconda3/bin/conda init bash
```

4. **Reload your shell configuration** to apply the changes:

```
source ~/.bashrc
```

5. **Clean up the installation file:**

```
rm miniconda_installer.sh
```

2.2 Prepare the Pipeline Configuration

The pipeline relies on a specific set of scientific and astronomical libraries. We utilize an `environment.yml` file to guarantee a reproducible and clean build.

1. Navigate to the root folder of your image processing project.
2. **Create a new configuration file:**

```
nano environment.yml
```

3. **Paste the following configuration** exactly as written:

```
name: pipeline_env
channels:
  - conda-forge
  - defaults
dependencies:
  - python=3.10
  - pip
```

```
- numpy
- scipy
- astropy
- ccdproc
- astroscrappy
- photutils
- astroalign
- scikit-image
- astroquery
- matplotlib
- pandas
- psutil
- tqdm
- pyyaml
- astrometry
```

Package roles: `astroquery` powers the Phase 3 reference-catalog queries (APASS / Pan-STARRS / SDSS); `pandas` and `pyyaml` back the catalog/config layer; and `astrometry` provides the `solve-field` binary used for plate solving in Phase 2. All other libraries are the numerical and image-processing core (`ccdproc`, `photutils`, `astroalign`, `astroscrappy`).

4. Save and exit the text editor.

2.3 HPC Network Drive Bypass (Critical for Servers)

Note: When installing Conda environments on High-Performance Computing (HPC) nodes or mounted network storage, the fast dependency solver often crashes with a `sqlite3.OperationalError:database is locked` error due to file-lock restrictions. To bypass this, temporarily route the Conda cache to the server's local storage.

1. **Ensure any existing locked cache directory is deleted.** (Replace the path with your actual Miniconda installation path if it differs):

```
rm -rf ~/miniconda3/pkgcache
```

2. **Create a temporary cache folder** in the server's fast local `/tmp` directory:

```
mkdir -p /tmp/pipeline_conda_cache
```

3. **Create a symbolic link** pointing the Conda cache to this fast local folder:

```
ln -s /tmp/pipeline_conda_cache ~/miniconda3/pkgcache
```

2.4 Build the Environment

With the configuration ready and the database lock bypassed, you can safely build the pipeline environment.

1. **Execute the creation command** using the configuration file:

```
conda env create -f environment.yml
```

2. Wait for the solver to resolve dependencies and download the packages.
3. **Safely remove the temporary cache bypass** to prevent future errors when the server automatically clears temporary files:

```
rm ~/miniconda3/pkgs/cache
mkdir -p ~/miniconda3/pkgs/cache
rm -rf /tmp/pipeline_conda_cache
```

4. Activate the newly created environment:

```
conda activate pipeline_env
```

2.5 Install the `cassa-photometry` Package

The pipeline ships as a single installable Python package (`cassa-photometry`). Cloning the repository and installing it in *editable* mode registers the command-line tools (`cassa-calibrate`, `cassa-integrate`, `cassa-photometry`, `cassa-diagnose`, `cassa-verify`, `cassa-run`) on your `PATH`.

1. Clone the repository and enter the pipeline directory:

```
git clone <repository-url>
cd photometric_pipeline
```

2. Install the package into the active environment.

Because the Conda environment already provides every scientific dependency, we pass `-no-build-isolation` (reuse the installed build tools) and `-no-deps` (do not re-resolve libraries against PyPI). This is also the correct approach on HPC nodes with restricted outbound network access.

```
pip install -e . --no-deps --no-build-isolation
```

Why editable? The `-e` flag links the installed command to the live source tree, so pulling updates from `git` takes effect immediately with no reinstall. Drop the `-e` for a fixed, production-frozen deployment.

2.6 Astrometry Index Files (Required for Phase 2)

The `solve-field` binary performs plate solving by matching star patterns against pre-computed *index files*. These files are large (several GB) and are **not** bundled with the package or the repository; they must be downloaded once for the field of view of your instrument.

1. **Download the appropriate index series** (e.g. the 4100–4200 series from the Astrometry.net data server) and place the `index-*.fits` files in a dedicated directory, for example `astrometry_data/`.
2. **Point the pipeline at that directory** using the `CASSA_Astrometry_INDEX` environment variable (highest priority), the `phase2.astrometry_index_dir` config key, or by leaving the files in a `./astrometry_data` folder relative to your working directory (the default):

```
export CASSA_Astrometry_INDEX=/path/to/astrometry_data
```

Choosing an index scale: Select index files whose scale spans your field of view. Small-FOV instruments (arcminutes) need the low-numbered, finely-scaled series, while wide-field optics use the higher-numbered series. Installing an unnecessarily broad range only slows the solver; it does not change the result.

2.7 Verification

To ensure all packages and C-compiled binaries built correctly for your specific Linux architecture, run these two tests:

1. **Verify Astrometry.net** is correctly mapped and accessible by the system:

```
solve-field --version
```

2. **Verify heavy astronomical Python libraries** load without conflict:

```
python -c "import astropy, ccdproc, astroalign, photutils, astroquery; print
('All core modules loaded successfully!')"
```

3. **Verify the installed package and its command-line tools:**

```
python -c "import cassa_photometry; print(cassa_photometry.__version__)"
cassa-calibrate --help
```

2.8 Pipeline Execution Guide

Once installation is complete and verified, you can process data through the console commands installed by the package. Always ensure the Conda environment is active first:

```
conda activate pipeline_env
export CASSA_ASTROMETRY_INDEX=/path/to/astrometry_data # needed by Phase 2
```

Command	Phase	Role
cassa-calibrate	1	Instrument Signature Removal → calibrated_*.fits
cassa-integrate	2	Alignment, stacking, WCS → Master_*.fits
cassa-photometry	3	Zero point, flux calibration, catalog
cassa-diagnose	4	PSF + visual diagnostics report
cassa-verify	—	Catalog check vs. APASS / Pan-STARRS / SDSS
cassa-run	1–3	All science phases, chained

2.8.1 Running Phase 1 (Instrument Signature Removal)

Phase 1 takes an input directory of raw FITS files (**-i**) and an output directory for the calibrated frames (**-o**). Each output is a multi-extension FITS file carrying **SCI**, **ERR**, and **DQ** planes (see Part III).

```
cassa-calibrate -i NGC7331_thairobotic/Raw -o NGC7331_thairobotic/cassa_output/
calibrated
```

2.8.2 Running Phase 2 (Integration & Astrometry)

Phase 2 ingests the calibrated frames, buckets them by target/filter/camera, stacks them with inverse-variance weighting, and plate-solves the result. It writes each master stack into a time-stamped **RunNN_*** sub-directory. Use **-yes** for non-interactive runs and **-keep-temps** to retain solver scratch files.

```
cassa-integrate NGC7331_thairobotic/cassa_output/calibrated --yes
```

2.8.3 Running Phase 3 (Photometry & Zero Point)

Phase 3 processes every master stack in the run directory: it cross-matches detected stars against reference catalogs, derives a filter-wise zero point with uncertainty, writes a flux-calibrated image and an error-carrying source catalog. It requires network access for the catalog queries.

```
cassa-photometry NGC7331_thairobotic/cassa_output/calibrated/Run01_*/
```

2.8.4 Running Phase 4 (Diagnostics)

Phase 4 validates the pipeline stage-by-stage. Point it at the Phase 2 run directory (it auto-discovers the calibrated, master, and catalog products) and supply the raw directory with `-raw` for the stage-0 checks. It writes `diagnostics/diagnostics_report.pdf`, per-stage PNGs, a `metrics.json`, and a health summary.

```
cassa-diagnose NGC7331_thairobotic/cassa_output/calibrated/Run01_*/ \
--raw NGC7331_thairobotic/Raw
```

2.8.5 End-to-End and Verification

`cassa-run` executes Phases 1–3 in sequence, and `cassa-verify` independently checks the output catalog against global reference catalogs:

```
# Chained: calibrate -> integrate -> photometry
cassa-run -i NGC7331_thairobotic/Raw -o NGC7331_thairobotic/cassa_output/
calibrated

# Independent photometric sanity check
cassa-verify NGC7331_thairobotic/cassa_output/calibrated/Run01_*/
```

2.8.6 Configuration Overrides

Every tunable parameter (cosmic-ray thresholds, stacking sigma-clip, aperture radii, zero-point match tolerance, saturation level, ...) has a documented default in `cassa_photometry.config`. Any subset can be overridden with a YAML file passed via `-config`:

```
# my_config.yaml
# phase1:
#   saturation_adu: 60000
# phase3:
#   fwhm: 4.0
#   zp_sigma_clip: 2.5
cassa-photometry NGC7331_thairobotic/cassa_output/calibrated/Run01_*/ --config
my_config.yaml
```

Part III: Image Processing Architecture

3 Pipeline Overview & Data Model

The pipeline is a single installable package (`cassa_photometry`) organised into four sequential phases, each exposed as a console command and each sharing one set of conventions (central configuration, unified logging, and a common FITS data model):

Phase	Command	Transformation
1. Calibration (ISR)	<code>cassa-calibrate</code>	Raw frames $\rightarrow$ calibrated frames (electrons)
2. Integration	<code>cassa-integrate</code>	Calibrated frames $\rightarrow$ WCS-solved master stacks
3. Photometry	<code>cassa-photometry</code>	Master stacks $\rightarrow$ zero point + flux-calibrated catalog
4. Diagnostics	<code>cassa-diagnose</code>	All stages $\rightarrow$ PSF + visual QA report

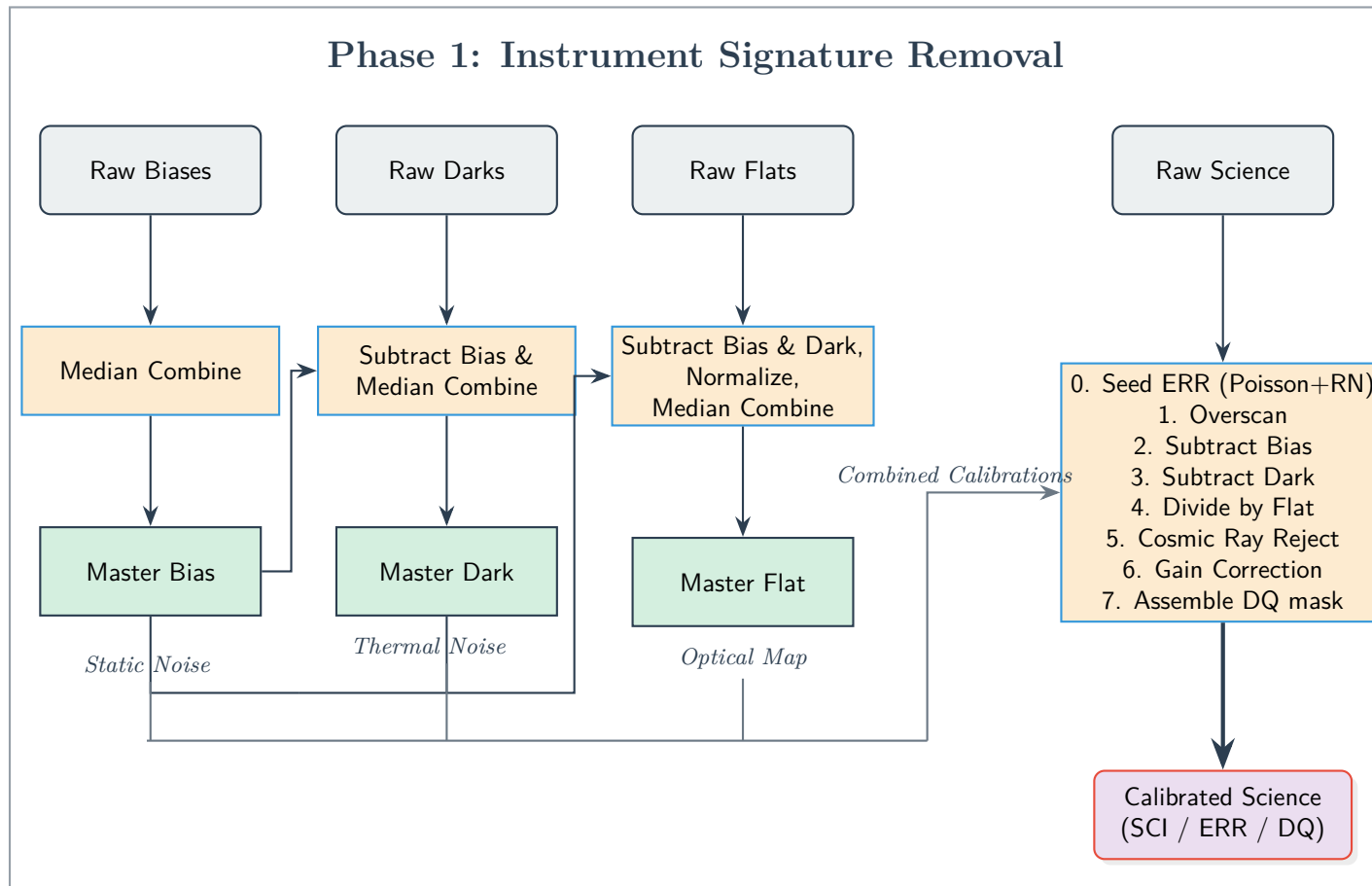
3.1 The Three-Plane Data Model (SCI / ERR / DQ)

The defining architectural principle of the pipeline is that **every science image carries a complete, per-pixel error budget from the very first calibration step through to the final catalogue**. Following the conventions of mature observatory pipelines (Astropy `ccdproc`, LCO BANZAI, Gemini DRAGONS), each product is a multi-extension FITS file with three planes:

- **SCI (Primary HDU)**: the science data itself — in electrons after Phase 1, or Janskys after flux calibration.
- **ERR**: the 1σ per-pixel uncertainty, in the same units as SCI. It is *seeded* in Phase 1 from the detector gain and read noise and mathematically *propagated* through every subsequent operation.
- **DQ**: an integer data-quality bitmask flagging saturated pixels (bit 1), bad/hot/dead pixels (bit 2), cosmic rays (bit 4), and no-data/NaN regions (bit 8).

Placing SCI in the primary HDU keeps the files compatible with ordinary tools (`fits.getdata`, DS9, `solve-field`), while the ERR and DQ extensions travel alongside. This error chain is what allows Phase 3 to report rigorous magnitude uncertainties and a zero point with a formal error bar. (See [Appendix A.12 for the error-seeding mathematics](#) and [Appendix A.13 for the DQ bitmask](#).)

4 Data Flow Architecture: Phase 1 (ISR)



5 Phase 1: Instrument Signature Removal (ISR)

Core Objective: To mathematically erase instrument-induced anomalies (static electronic noise, thermal currents, and optical defects) from raw astronomical data, outputting pristine calibrated frames — each with a seeded **ERR** plane and a **DQ** bitmask — ready for integration.

5.1 Master Calibration Processes

Phase 1 relies on constructing high Signal-to-Noise Ratio (SNR) master calibration frames using rigorous statistical rejection to prevent noise injection during science calibration. Critically, **each master frame is built with its own uncertainty** (the standard error of the robust combine, $\text{mad_std}/\sqrt{N}$). Because `ccdproc` propagates uncertainty in quadrature through every arithmetic operation, this calibration noise automatically folds into the science frame's **ERR** plane during bias/dark subtraction and flat division.

5.1.1 Master Bias Generation (Median Combine)

- **Process:** The pipeline groups all raw zero-second exposure frames. These frames capture the fixed electronic readout noise floor of the sensor ("Static Noise").
- **Mathematics:** The 2D arrays are stacked into a 3D data cube. The algorithm calculates the **Median** value through the Z-axis for every (X,Y) pixel coordinate. This effectively models the read-noise floor while mathematically rejecting transient anomalies like cosmic rays that only appear in a single frame.

5.1.2 Master Dark Generation (Subtract Bias & Median Combine)

- **Process:** Raw dark frames capture the "Thermal Noise" that accumulates in the silicon matrix over time, even with the shutter closed.
- **Mathematics:** The pipeline subtracts the newly generated **Master Bias** from each individual Raw Dark frame. This ensures we are isolating purely thermal current and not double-counting read noise. The bias-subtracted darks are then median-combined along the Z-axis to reject transients, outputting the clean **Master Dark**.

5.1.3 Master Flat Generation (Subtract, Normalize, Median Combine)

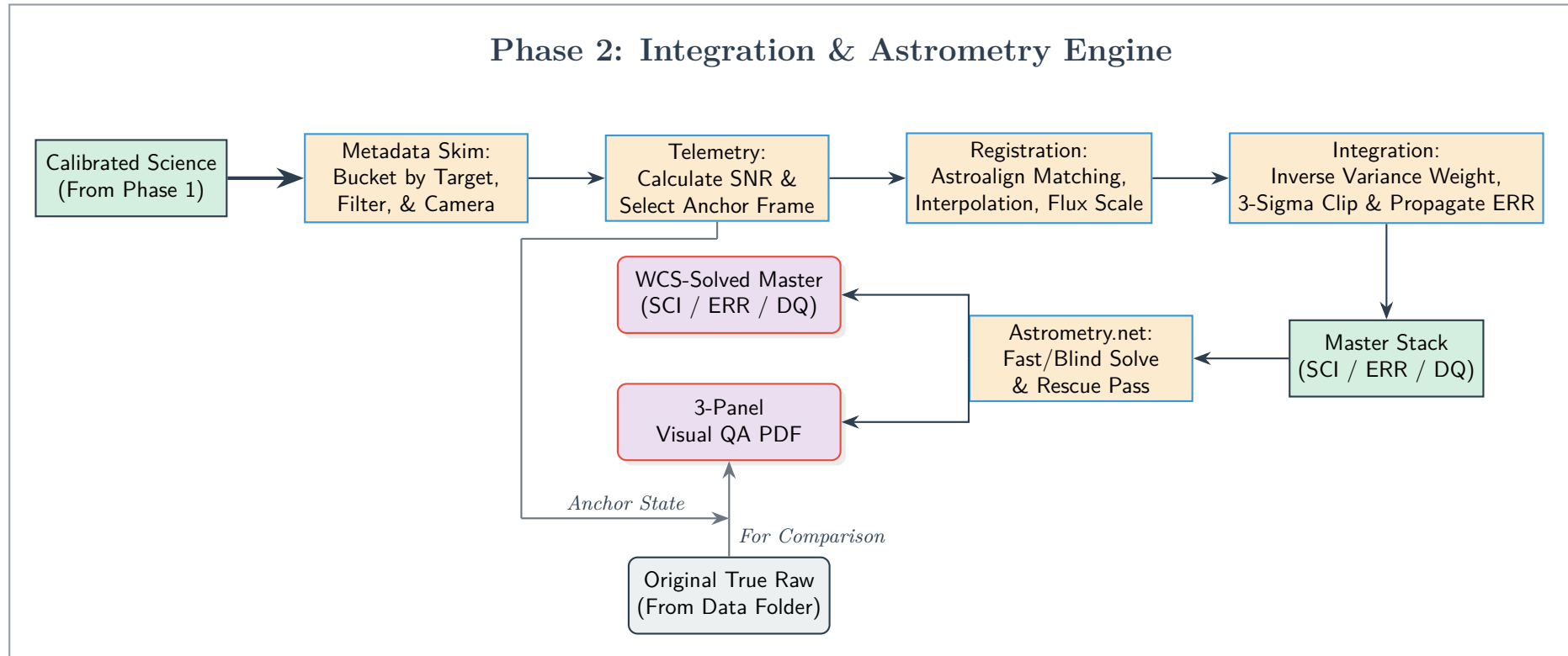
- **Process:** Raw flats map the physical optical imperfections of the telescope (vignetting, dust donuts).
- **Mathematics:**
 1. **Subtraction:** The Master Bias and Master Dark are subtracted from every raw flat to remove their electronic and thermal signatures.
 2. **Normalization:** The scalar median ADU (Analog-to-Digital Unit) value of the subtracted flat is calculated. The entire 2D matrix is then divided by this scalar value. This forces the array's average background to center around 1.0.
 3. **Combination:** The normalized frames are median-combined, producing an "Optical Map" of percentage-based multipliers. (See [Appendix A.2 for a detailed breakdown of the normalization mathematics](#))

5.2 The Science Processing Flow

Raw science frames are fed through the `UniversalProcessor` and subjected to a strict sequence of mathematical corrections. The very first act is to *seed* the error budget; from that point on the `ERR` plane is carried through every step automatically:

1. **Error Budget Seeding:** As each raw frame is loaded, the pipeline attaches a 1σ uncertainty computed from the detector gain and read noise, $\sigma_{\text{ADU}} = \sqrt{gI + \sigma_R^2} / g$ (Poisson photon noise plus read noise; `ccdproc.create_deviation`). A saturation mask is also recorded from the raw ADU values for later DQ flagging. (See [Appendix A.12](#))
2. **Overscan Correction (Conditional):** If the sensor has an unilluminated overscan region, the engine calculates the median row-by-row noise floor and subtracts it to correct for amplifier drift during readout. Because these extreme edge pixels are physically masked from light, they record the exact baseline electronic noise present at the fraction of a second that specific row is processed. Subtracting this unique median from each row completely eliminates horizontal banding caused by the camera’s amplifier heating up or fluctuating during the download sequence. (See [Appendix A.3 for a deep dive into banding correction](#))
3. **Bias Subtraction:** The Master Bias (Static Noise) is subtracted from the science frame; its uncertainty adds in quadrature to the `ERR` plane.
4. **Dark Subtraction:** The Master Dark (Thermal Noise) is subtracted from the science frame. The pipeline strictly enforces a 1:1 exposure match (`scale=False`) to preserve non-linear thermal structures. (See [Appendix A.6 for the physics of dark scaling](#))
5. **Flat Division:** The image matrix is divided pixel-by-pixel by the normalized Master Flat, which mathematically brightens vignetted corners and erases dust shadows. The flat’s relative uncertainty propagates into `ERR`.
6. **Cosmic Ray Rejection:** The pipeline executes the `astroscrappy` Laplacian edge-detection algorithm. By analyzing the specific Gain and Read Noise of the camera, it differentiates legitimate point-sources (stars) from high-energy cosmic ray strikes, masking and interpolating the damaged pixels; the affected pixels are flagged in the DQ plane. (See [Appendix A.4 for the underlying mathematics of the Laplacian filter](#))
7. **Gain Correction:** The final 2D matrix is multiplied by the hardware gain. This converts the data *and its uncertainty* from arbitrary ADUs into true, physical Electrons (e^-), and the BUNIT FITS header is updated accordingly. (See [Appendix A.1 for how hardware metadata is safely extracted](#))
8. **DQ Assembly & Output:** A single integer DQ bitmask is assembled from the saturation mask, the flat-derived Bad-Pixel Mask (BPM), the cosmic-ray mask, and any NaN/no-data pixels. The calibrated frame is written as a multi-extension FITS file with SCI (electrons), `ERR` (1σ), and DQ planes. (See [Appendix A.5 for BPM generation](#) and [Appendix A.13 for the bitmask](#))

6 Data Flow Architecture: Phase 2 (Integration & Astrometry)



7 Phase 2: Integration, Astrometry & QA Engine

Core Objective: To automatically ingest Phase 1 frames, mathematically align them to a dynamic anchor, integrate them using inverse-variance weighting and sigma-clipping, and map the deep stack to true celestial coordinates.

7.1 Grouping and Telemetry Engine

7.1.1 Metadata Skim (Target Grouping)

The `FITSHandler` reads the headers of incoming calibrated science frames. It extracts the `OBJECT`, `FILTER`, and hardware metadata (focal length, pixel size). Files are dynamically bucketed into `TargetGroup` objects ensuring only data matching the exact Object/Filter/Camera combination are stacked together.

7.1.2 Telemetry (Anchor Selection)

Instead of randomly picking an alignment reference, the pipeline extracts a 2D background model via `photutils`. It calculates the `mad_std` (Median Absolute Deviation) to determine the true noise floor. The single frame with the highest 99th-percentile signal divided by the lowest noise floor is crowned the **Anchor Frame**. (See [Appendix A.8 for why MAD is used for noise estimation](#))

7.2 Integration Engine

7.2.1 Geometric Registration & Flux Scaling

- **Astroalign Matching:** The pipeline detects point sources and maps asterism triangles between the target frame and the Anchor frame. It solves an affine transformation matrix (Translation, Rotation, Scale). (See [Appendix A.9 for the logic of geometric hashing](#))
- **Interpolation:** The target image is geometrically warped to perfectly overlay the anchor using 3rd-order (bicubic) interpolation to preserve sub-pixel stellar profiles during fractional pixel shifts. (See [Appendix A.7 for an explanation of sub-pixel math](#))
- **Flux Scaling:** The `MathEngine` calculates the median flux ratio of the matched stars. The warped target is multiplied by this scalar, normalizing its brightness to match the Anchor (correcting for passing cirrus clouds or transparency loss).
- **Error-Plane Registration:** The ERR plane travels with the science data — its variance is warped by the same transform and scaled by the square of the flux ratio ($\sigma^2 \rightarrow s^2\sigma^2$) — so the propagated uncertainty of every pixel remains valid after alignment.

7.2.2 Stacking (Inverse Variance Weighting & 3-Sigma Clipping)

- **Inverse Variance Weighting:** The noise variance of every aligned frame is calculated. Frames with lower noise are given heavier mathematical influence in the final stack; noisy frames are mathematically suppressed.
- **3-Sigma Clipping:** The 3D data cube undergoes Z-axis sigma-clipping. For every pixel coordinate, the algorithm computes the median and standard deviation through the stack. Any pixel deviating by more than 3-sigma (e.g., satellite trails, airplanes) is masked and rejected before the final weighted mean is computed, outputting the **Master Stack**. (See [Appendix A.10 for the mathematics of integration](#))

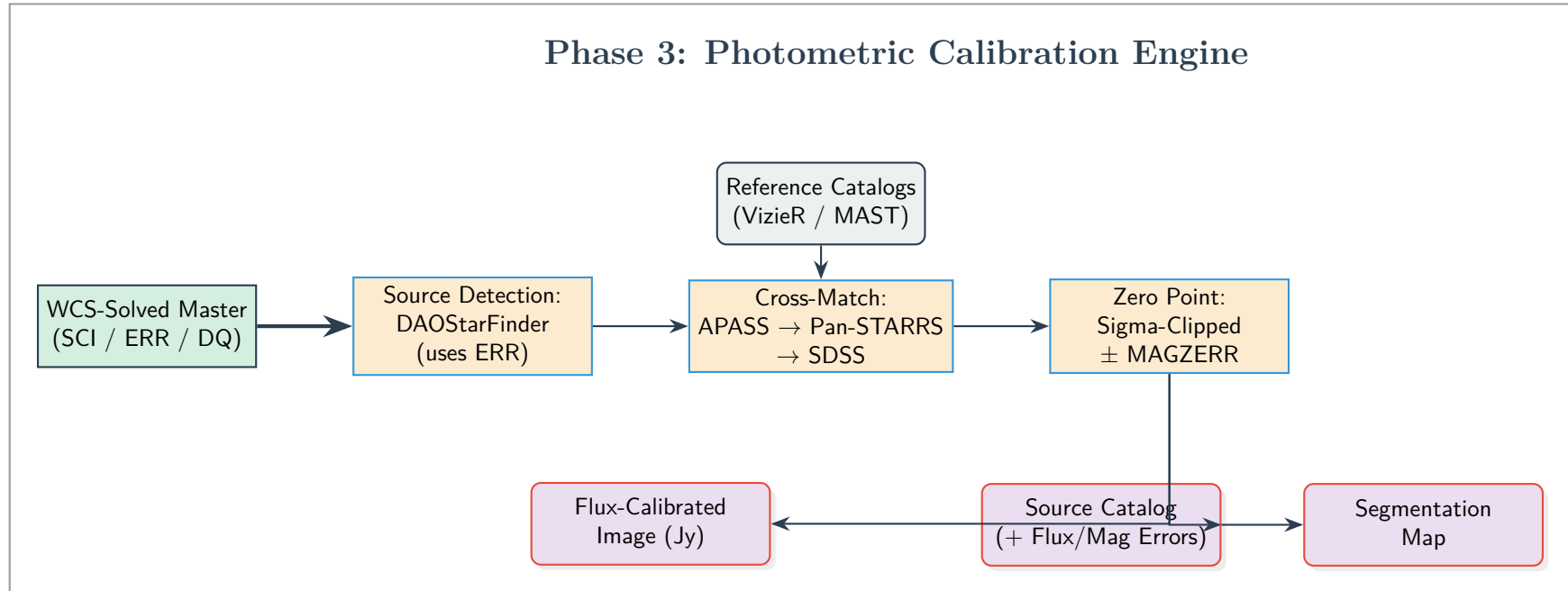
- **Propagated Master ERR:** The variance of the weighted mean, $\text{Var} = \sum w_i^2 \sigma_i^2 / (\sum w_i)^2$ over the surviving frames, is *materialized* as the master's **ERR** plane rather than merely logged. A pixel's **DQ** is flagged where no frame contributed (NaN). The historical fixed "+100 ADU" display pedestal has been removed, so the stack remains on a physical electron scale suitable for absolute photometry.

7.3 Astrometry Engine

7.3.1 Plate Solving & Rescue Pass

- **Fast/Blind Solve:** The **WCSSolver** passes the Master Stack to the local Linux **solve-field** binary. It injects a physical **PIXSCALE** estimate (derived from focal length and pixel size) to restrict the search radius for a highly optimized "Fast Solve."
- **Rescue Pass:** Narrowband images (like H-Alpha) often lack enough stars to plate-solve. If a solve fails, the engine borrows the exact Right Ascension and Declination from a successfully solved filter within the same **TargetGroup**, executing a forced local search to rescue the astrometry. (See [Appendix A.11 for plate solving optimization](#))
- **Error-Plane Preservation:** **solve-field** only understands a single-image FITS and drops extra extensions from its output. The **WCSSolver** therefore solves on the **SCI** plane, then re-attaches the **ERR** and **DQ** planes to the WCS-solved result — so the master remains a full three-plane multi-extension file after astrometry.

8 Data Flow Architecture: Phase 3 (Photometry & Zero Point)



9 Phase 3: Photometry, Zero Point & Catalogs

Core Objective: To place the deep master stacks on a physical, filter-wise photometric scale — deriving a zero point *with a formal uncertainty*, converting pixels to Janskys, and producing a source catalogue in which every magnitude carries an error bar.

9.1 Reference-Catalog Cross-Match

9.1.1 Priority Cascade

The engine reads the WCS to convert detected star positions into sky coordinates, then queries global reference catalogues in a filter-aware priority cascade: **APASS** (Johnson B/V and Sloan $g'/r'/i'$, ideal for small-telescope depths), falling back to **Pan-STARRS DR2** and then **SDSS DR12**. The first catalogue to return valid stars for the requested band wins, and its per-star magnitude *and its uncertainty* are standardised into a common table. (See [Appendix A.14 for the zero-point mathematics](#))

9.2 Filter-Wise Zero Point

9.2.1 Differential Photometry with Uncertainty

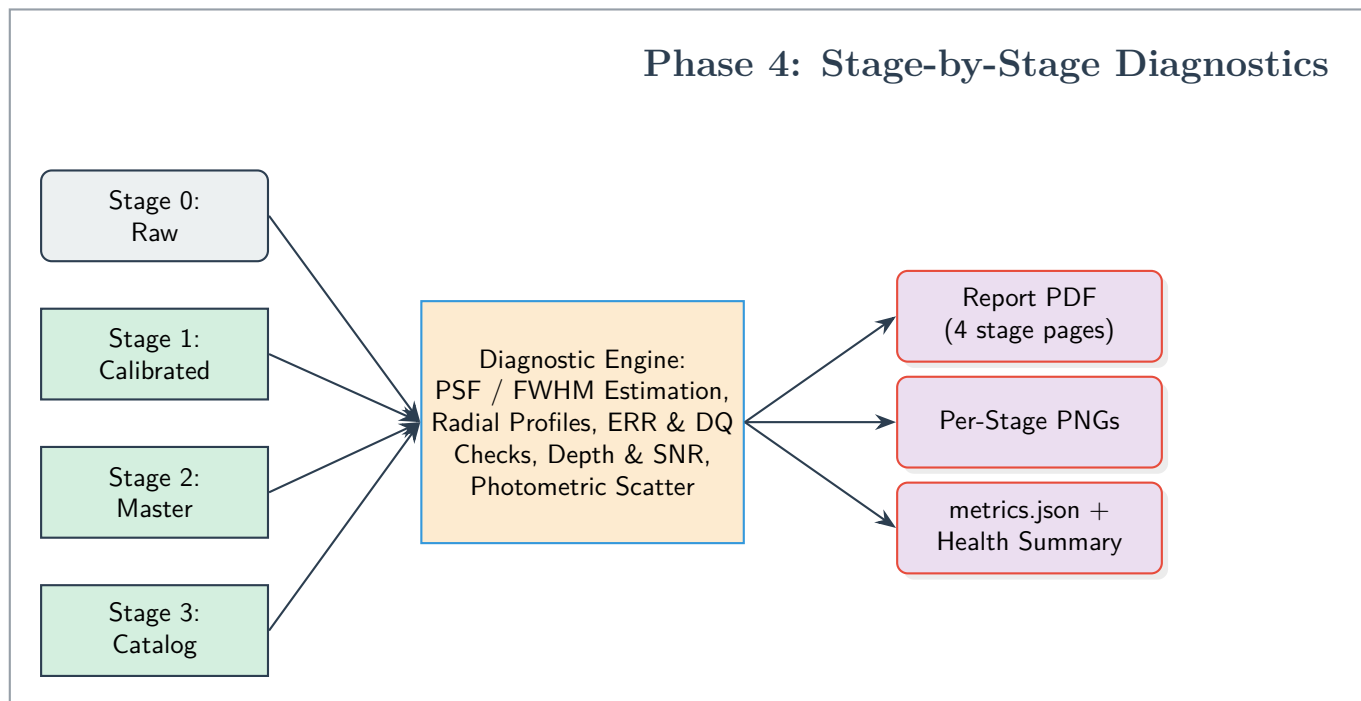
- **Instrumental Measurement:** For each catalogue-matched star the engine performs aperture photometry using the ERR plane, yielding an instrumental magnitude and its error, $\delta m_{\text{inst}} = 1.0857 \delta F / F$.
- **Per-Star Offset:** Each star gives a zero-point estimate $ZP_i = m_{\text{cat}} - m_{\text{inst}}$, with an uncertainty combining the instrumental and catalogue errors in quadrature.
- **Robust Combination:** The per-star offsets are sigma-clipped and combined with an inverse-variance weighted mean. The pipeline records **MAGZERO**, the zero-point uncertainty **MAGZERR** (the larger of the formal weighted error and the standard error of the surviving scatter), and **NZPSTARS** in the FITS header — one zero point per filter, each with a defensible error bar.

9.3 Products

9.3.1 Flux Calibration & Catalog

- **Flux-Calibrated Image:** Pixels are converted to Janskys via the AB relation $F_\nu = 3631 \times 10^{-ZP/2.5}$; the ERR plane is scaled by the same factor and preserved.
- **Source Catalog:** A deblended segmentation catalogue is measured with the ERR plane, exposing **Flux_Error**, **Mag_Error** $= \sqrt{(1.0857 \delta F / F)^2 + \text{MAGZERR}^2}$, **SNR**, and **DQ FLAGS** columns alongside positions, magnitudes, and shape parameters.
- **Verification:** The independent **cassa-verify** tool re-queries reference catalogues and reports the residual offset and scatter, confirming the reported **MAGZERR** is realistic.

10 Data Flow Architecture: Phase 4 (Diagnostics)



11 Phase 4: Diagnostics & Validation

Core Objective: To verify that the pipeline is behaving correctly at every stage, by estimating the point-spread function and producing quantitative, visual quality-assurance for the raw frame, the calibrated frame, the WCS-solved master, and the photometric catalogue.

11.1 PSF & Image Quality

Across all stages, the diagnostic engine detects isolated bright stars, fits each with a 2D Gaussian, and reports a robust median **FWHM** (in pixels and, where a plate scale exists, arcseconds), an ellipticity, and a stacked **radial profile**. A stable FWHM across stages confirms that calibration and stacking have not degraded the stellar profiles. (See [Appendix A.15 for the FWHM fitting method](#))

11.2 Per-Stage Checks

- **Stage 0 (Raw):** image and histogram, background level, detected-star overlay, and a near-saturation map.
- **Stage 1 (Calibrated):** SCI/ERR/DQ panels, DQ flag counts, an *error-vs-signal* (Poisson) check, and the background-flatness improvement relative to the raw frame.
- **Stage 2 (Master):** SCI/ERR panels and an SNR map, the depth boost relative to a single frame (the noise should drop by $\approx \sqrt{N}$), and the WCS status with field centre and plate scale.
- **Stage 3 (Photometry):** magnitude-error versus magnitude, SNR versus magnitude, number counts / limiting magnitude, a spatial source map, and a star/galaxy shape split, together with the recorded zero point and MAGZERR.

11.3 Outputs

The engine auto-discovers the products of a Phase 2 run directory (pairing them by filter) and writes a multi-page `diagnostics_report.pdf`, per-stage PNGs, a machine-readable `metrics.json`, and a **pipeline-health summary** marking each stage OK, SKIP, or FAIL. Stages whose products are absent degrade gracefully rather than aborting the report.

12 Appendix: Technical Implementation Details

12.1 Dynamic FITS Header Extraction (The Adapter Pattern)

Astronomical FITS headers lack strict standardization across observatories (e.g., gain might be stored as `EGAIN`, `GAIN`, or `SYSGAIN`). To prevent pipeline crashes, Phase 1 relies on the **Adapter Pattern** via an `InstrumentProfile` class.

- **Translation Layer:** The core mathematical engine never directly reads a FITS file. Instead, it queries the `InstrumentProfile`. The profile safely iterates through known header variants to extract critical data like Gain and Read Noise.
- **Failsafe Fallbacks:** In the event of missing or corrupted headers, the profile utilizes a hard-coded `hardware_database` dictionary. By extracting the telescope ID, it can safely inject verified baseline hardware constants (e.g., `'T32': {'gain': 1.0, 'read_noise': 9.0}`), ensuring the processing loops never fail due to missing metadata.

12.2 The Mathematics of Flat Frame Normalization

Simply averaging flat frames together is mathematically dangerous, as a brighter flat frame would overpower a dimmer one, and transient anomalies (like a cosmic ray striking the flat panel) would be permanently baked into the Master Flat.

- **The Percentage Map:** By dividing every single pixel in a flat frame by that frame's overall median value, the arbitrary brightness (ADU) is converted into a percentage map centered around 1.0. A darkened corner pixel might become 0.80 (meaning 80% transmission).
- **Statistical Rejection:** Once normalized, the stack of flat frames is merged using a **Median Combine** along the Z-axis. If a cosmic ray artificially spikes a pixel to a massive value in one frame, the median calculation entirely ignores it, resulting in a pristine Master Flat that solely maps permanent optical defects.

12.3 Overscan Banding Correction

Amplifier heat and voltage fluctuations cause the baseline electronic noise to drift as a sensor is read out from top to bottom, resulting in horizontal banding.

- **Row-by-Row Correction:** The overscan region is a set of physical pixels explicitly covered by an opaque mask. The pipeline isolates this region and calculates a unique median noise value for *each individual row* (`axis=1`). By subtracting this dynamically shifting baseline from the corresponding active pixels in that exact same row, the amplifier drift is completely mathematically flattened.

12.4 Laplacian Cosmic Ray Detection (`astrocrappy`)

The pipeline utilizes an adaptation of the L.A.Cosmic algorithm to differentiate between legitimate celestial point-sources and radiation strikes.

- **Second Derivative Filter:** Legitimate stars possess a Point Spread Function (PSF)—their light blurs across multiple pixels in a smooth Gaussian curve. Cosmic rays impact the silicon directly, creating an infinitely sharp, single-pixel spike. The pipeline applies a **Laplacian filter** (calculating the second derivative of brightness), which measures the *sharpness* of the data.

- **Physics-Based Thresholding:** The algorithm is fed the specific `EGAIN` and `READNOISE` of the camera to establish the true statistical noise floor. If a pixel's Laplacian sharpness exceeds a strict threshold (e.g., 4.5σ above the noise limit), it is mathematically proven to be a cosmic ray. The pixel is masked, and its value is interpolated from surrounding healthy pixels.

12.5 Bad Pixel Mask Generation (BPM)

While cosmic rays are transient anomalies, camera sensors also suffer from permanent hardware defects, such as "dead" pixels (always read zero) or "hot" pixels (always read maximum signal).

- **Derivation from Flats:** The pipeline *generates* the Bad Pixel Mask directly from the normalised master flats: any pixel whose response deviates strongly from unity (by default, below $0.5\times$ or above $1.5\times$ the median), or which is non-finite, is flagged as defective. Pooling this across all filters yields a robust, detector-level BPM.
- **Folded into DQ:** Rather than silently discarding these pixels, the BPM is recorded in the DQ bitmask (bit 2, `BAD_PIXEL`) so that downstream stages can see, and choose how to treat, every flagged coordinate.

12.6 Strict Dark Current Matching vs. Scaling

Astronomical thermal noise is notoriously complex. While baseline dark current accumulates somewhat linearly over time, other thermal artifacts do not.

- **The Scaling Problem:** Modern CMOS sensors (and many CCDs) exhibit "amplifier glow"—a localized thermal bloom near the sensor's electronics. This glow is highly non-linear. If the pipeline were to mathematically scale a 300-second dark frame by a factor of 2 to calibrate a 600-second science frame, it would severely over-correct the amplifier glow, creating black holes in the final image.
- **The `scale=False` Imperative:** By explicitly setting `scale=False` during the `subtract_dark` operation, the pipeline enforces strict rigor: it refuses to scale thermal frames. It mandates that the Master Dark exposure time must perfectly match the science frame exposure time, preserving the non-linear thermal structures and ensuring a mathematically perfect subtraction.

12.7 Sub-Pixel Interpolation Physics

When aligning multiple astronomical exposures, stars never land on the exact same physical pixels due to atmospheric turbulence or mechanical tracking errors. To correct for these fractional shifts (e.g., sliding an image by 0.4 pixels), the pipeline must mathematically guess the new grid values.

- **The Danger of Low-Order Math:** A star's light spreads across pixels in a smooth Gaussian bell curve. Using 1st-Order (Bilinear) interpolation essentially draws straight lines between neighboring pixels. During a sub-pixel shift, this straight-line math literally cuts off the peak of the star's bell curve, resulting in a blunt profile and a permanent loss of scientific flux data.
- **3rd-Order Preservation:** By calculating smooth, 3rd-order polynomial curves across a 4×4 grid of neighboring pixels, the bicubic interpolation algorithm perfectly recreates the natural, curved slope of the star. This allows the pipeline to slide images by microscopic fractions of a pixel while mathematically preserving the exact shape and peak brightness of the astronomical data.

12.8 Robust Noise Estimation (Median Absolute Deviation)

When selecting the best Anchor frame, the pipeline must calculate the Signal-to-Noise Ratio (SNR) of every image. However, calculating standard deviation on an astronomical image is dangerous because bright stars and cosmic rays artificially inflate the variance.

- **MAD vs. Standard Deviation:** Instead of standard deviation, the pipeline uses the Median Absolute Deviation (`mad_std`). MAD calculates the median of the data, subtracts that median from every pixel, and then finds the median of those absolute differences. This makes the noise floor calculation entirely immune to bright outlier pixels, providing a mathematically pure representation of the true background sky noise.

12.9 Asterism Geometric Hashing (Astroalign)

Professional pipelines cannot rely on FITS header coordinates (WCS) for alignment because telescopes suffer from mechanical flexure, tracking drift, and meridian flips.

- **Triangle Invariance:** The `astroalign` algorithm extracts the brightest stars in an image and forms them into sets of triangles (asterisms). The internal angles of a triangle never change, regardless of whether the image is translated, scaled, or rotated 180°. By cross-matching these triangle ratios between the Target and the Anchor, the pipeline can blindly calculate a perfect affine transformation matrix without needing any telescope telemetry.

12.10 Integration Mathematics (Weighting & Clipping)

Combining the aligned images requires protecting the stack from transient events (satellites) and varying atmospheric conditions (passing clouds).

- **Inverse Variance Weighting:** Not all frames are equal. A frame taken through thin cirrus clouds will have higher noise. The pipeline assigns a weight to each pixel equal to $1/\sigma^2$. This ensures pristine frames mathematically dominate the final stack, while noisy frames contribute proportionally less, maximizing the final SNR.
- **Z-Axis Sigma Clipping:** To erase satellite trails, the algorithm looks "down" through the stack of aligned images at a specific (X, Y) coordinate. It calculates the median and standard deviation of that specific pixel column. If a satellite crossed that pixel in frame 4, its value will spike $> 3\sigma$ above the median. The algorithm mathematically masks that outlier before calculating the final weighted average.

12.11 Blind Plate Solving Optimization

Astrometry.net maps pixels to physical sky coordinates (Right Ascension / Declination) by comparing star patterns to a global index.

- **Speeding up the Blind Solve:** A true "blind" solve searches the entire sky at every possible magnification scale, which is computationally expensive. The pipeline extracts the camera's pixel size and telescope focal length from the FITS header to calculate a physical `PIXSCALE` (arcseconds per pixel). Injecting this constraint restricts the solver's search radius, turning a 5-minute blind solve into a 5-second localized solve.

12.12 Seeding the Error Budget (`create_deviation`)

The single most important step for producing science-grade uncertainties is to establish a physically correct error at the very start, before any subtraction can hide it.

- **The CCD Noise Equation:** A raw pixel’s variance is the sum of two independent physical sources: Poisson (shot) noise from the arriving photons, which in electrons equals the signal itself, and the fixed read noise of the amplifier. Working in ADU with gain g (e^-/ADU) and read noise σ_R (e^-), the per-pixel 1σ deviation is $\sigma_{\text{ADU}} = \sqrt{gI + \sigma_R^2} / g$.
- **Automatic Propagation:** Once this uncertainty is attached (`ccdproc.create_deviation`), every subsequent `ccdproc` operation — bias subtraction, dark subtraction, flat division, gain correction — combines it in quadrature with the master frames’ own uncertainties, so no bespoke error bookkeeping is required. Gain correction multiplies both the data and its uncertainty by g , converting the whole budget from ADU to electrons.

12.13 The Data-Quality Bitmask (DQ)

Rather than deleting or interpolating away suspect pixels invisibly, the pipeline records *why* each pixel is suspect in an integer bitmask, so downstream code can make its own decisions.

- **Independent Bits:** Each condition owns one bit — SATURATED (1), BAD_PIXEL (2), COSMIC_RAY (4), NO_DATA (8) — so multiple conditions can coexist on the same pixel via a bitwise OR. A pixel flagged 5 is both saturated and a cosmic-ray hit.
- **Propagation:** The DQ plane travels with SCI/ERR through stacking and photometry. In the source catalogue, the FLAGS column reports the OR of all DQ bits falling within each source’s segment, so a user can trivially reject, say, any source touching a saturated or bad pixel.

12.14 Filter-Wise Zero Point with Uncertainty

A zero point translates instrumental magnitudes onto a standard system; reporting it *without* an uncertainty makes every downstream magnitude untrustworthy.

- **Per-Star Estimates:** Each catalogue-matched star yields $\text{ZP}_i = m_{\text{cat}} - m_{\text{inst}}$. Because the instrumental measurement uses the ERR plane, each estimate carries its own uncertainty $\sqrt{\delta m_{\text{inst}}^2 + \delta m_{\text{cat}}^2}$, where the magnitude error relates to the fractional flux error by the Pogson factor $\delta m = 1.0857 \delta F / F$.
- **Robust Combination:** The set of ZP_i is sigma-clipped to reject mismatches and variables, then combined as an inverse-variance weighted mean. The reported MAGZERR is the larger of the formal weighted error $\sqrt{1 / \sum w_i}$ and the standard error of the surviving scatter $\sigma / \sqrt{N}$ — a deliberately conservative choice that captures both measurement precision and real field-to-field spread.

12.15 PSF Estimation via 2D Gaussian Fitting

The full-width at half-maximum (FWHM) of stars is the single most informative image-quality metric, tracking seeing, focus, and tracking errors.

- **Star Selection:** The engine detects sources, then keeps only isolated, non-edge, unsaturated stars (a saturated star has a flat top — many pixels sharing the peak value — and is rejected). This prevents blends and saturated cores from biasing the measurement.
- **Fitting & Robust Median:** Each selected star is fit with a 2D Gaussian plus a constant background; the FWHM follows from the fitted standard deviations as $2\sqrt{2 \ln 2} \sigma$, and the axis ratio yields an ellipticity. The per-star FWHMs are sigma-clipped and the median is reported, while the peak-normalised cutouts are stacked into a mean radial profile for visual inspection.

Part IV: Atmospheric Seeing Monitor

13 DIMM Overview & Deployment

The `cassa-dimm` package is a professional **Differential Image Motion Monitor (DIMM)**: a continuous atmospheric-seeing monitor that runs alongside the imaging pipeline. It drives an 8-inch SkyWatcher on an EQ6R-Pro fitted with a two-hole aperture mask, one hole carrying a wedge prism, so that every star produces *two* spots on the detector. By tracking how the separation between the two spots jitters from frame to frame, the pipeline measures the Fried parameter r_0 and reports the seeing — independently of telescope tracking errors, which cancel in the differential measurement.

13.1 Principle of Operation

A single-aperture image dances on the detector under atmospheric turbulence (and mount tracking error). Because both prism spots are formed by the *same* telescope, any motion common to both — tracking drift, wind shake, overall image wander — cancels when we look at the **difference** of the two centroids. What remains is the differential wavefront tilt between the two mask sub-apertures, whose variance is a clean, calibratable measure of the turbulence integrated along the line of sight. The variance is converted to r_0 and then to a seeing FWHM. (See [Appendix B.1](#))

13.2 Installation

The DIMM pipeline shares the same Conda environment as the imaging pipeline (Part II); only the package itself needs to be installed:

```
conda activate pipeline_env
cd DIMM
pip install -e . --no-deps --no-build-isolation
```

This registers three commands:

Command	Role
<code>cassa-dimm-monitor</code>	Continuous watch-folder seeing monitor (the production tool)
<code>cassa-dimm-batch</code>	One-shot reduction of a FITS cube or a folder of frames
<code>cassa-dimm-sim</code>	Generate synthetic frames with a known injected seeing (testing)

13.3 Running the Monitor

The acquisition script on the telescope server streams individual FITS frames into a watched folder. The monitor keeps a rolling window of the most recent frames and emits a fresh seeing estimate every few seconds. Copy and edit the example configuration first — in particular the **site latitude/longitude** (required for the airmass correction) and the watch folder:

```
cp config.example.yaml config.yaml      # set site coords + watch folder
cassa-dimm-monitor --config config.yaml --folder /path/to/watch_folder
```

Every measurement is written to the output directory:

File	Contents
<code>seeing_log.csv / .jsonl</code>	Time series: timestamp, seeing (zenith & raw), longitudinal/transverse, r_0 , airmass, N frames, N stars, star scatter, SNR, QC flags
<code>status_latest.json</code>	The latest measurement, for an observatory dashboard to poll
<code>seeing_timeseries.png</code>	A live seeing-versus-time plot

13.4 Trying It with Simulated Data

Before any real data is available, the whole chain can be exercised with the built-in simulator, which injects a *known* seeing so the result can be checked. The quickest test is a one-shot batch reduction:

```
# Generate 500 frames with an injected seeing of 1.5" (3 star doublets)
cassa-dimm-sim --out /tmp/dimm_demo --seeing 1.5 --nstars 3 --frames 500

# Reduce them -> should report ~1.5" with stars=3
cassa-dimm-batch /tmp/dimm_demo
# Seeing (zenith): 1.46" +/- 0.05 | raw=1.45" | r0=6.9 cm | stars=3 | flags
=[]
```

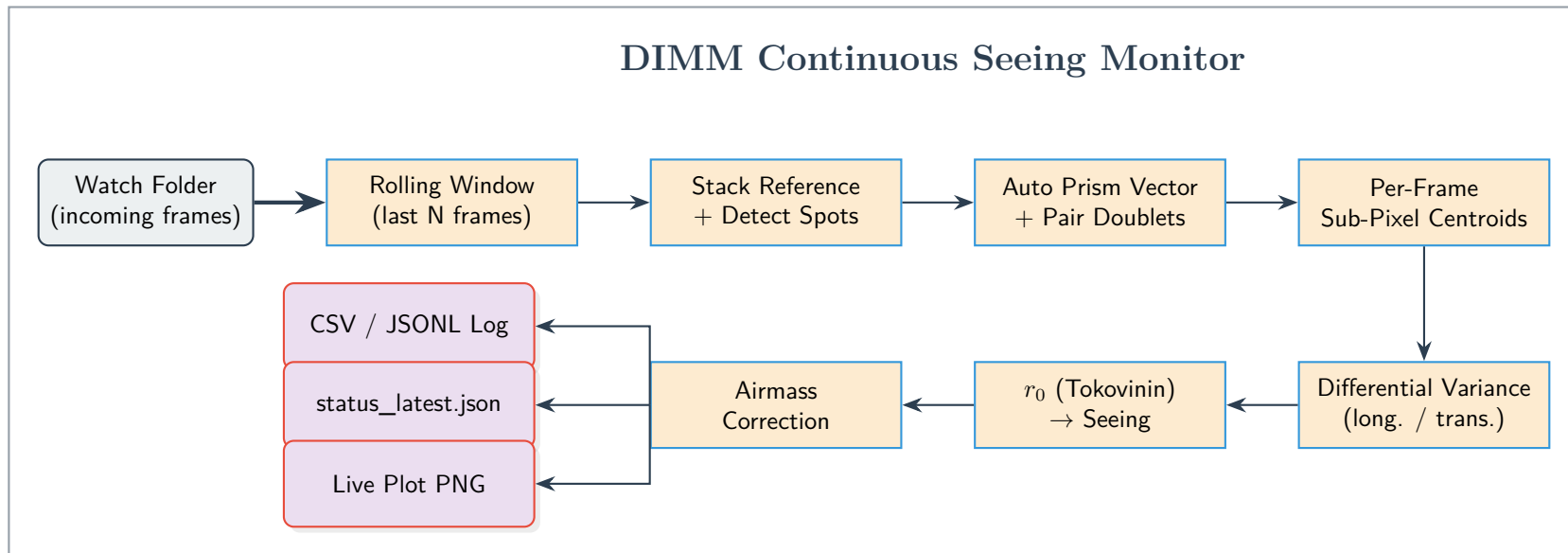
To exercise the *continuous* monitor and watch the outputs build up, point it at a folder and let frames appear there. In production the acquisition server streams frames into this folder; for a demo you can pre-fill it (and set `watch.process_existing: true`), or copy frames in a few batches to see the time series grow:

```
mkdir -p /tmp/dimm_watch /tmp/dimm_out
cassa-dimm-monitor --config config.yaml --folder /tmp/dimm_watch --outdir /tmp/
dimm_out &

# Feed simulated frames into the watched folder (repeat, or copy in batches)
cassa-dimm-sim --out /tmp/dimm_watch --seeing 1.5 --nstars 3 --frames 600
```

Then inspect `/tmp/dimm_out`: `status_latest.json` holds the most recent seeing, `seeing_log.csv` accumulates the time series, and `seeing_timeseries.png` is the live plot. A healthy run reports the longitudinal and transverse seeing in agreement and recovers the injected value.

14 Data Flow Architecture: DIMM Seeing Monitor



15 DIMM Reduction Engine

Core Objective: To turn a rolling stream of short-exposure frames — each showing every star twice — into a robust, airmass-corrected seeing value, using *all* the stars in the field for the best possible statistics.

15.1 Multi-Star Detection & Doublet Pairing

Unlike a classic single-star DIMM, the field may contain several stars, each split into a doublet by the prism.

- **Stacked Reference:** The frames in the current window are mean-combined into a high-SNR reference; `DAOStarFinder` then detects all spots on it.
- **Prism-Vector Auto-Calibration:** The prism shifts every star’s second image by the *same* fixed vector. The engine histograms all pairwise spot offsets; the one repeated offset that many pairs share is the prism doublet vector. (A known value can be pinned in the config instead.)
- **Pairing:** Spots are greedily paired — brightest first — into doublets matching that vector, rejecting blends, saturated cores, and edge sources. Each surviving doublet’s two reference positions are then tracked frame-to-frame.

15.2 Sub-Pixel Centroiding

For each spot in each frame the engine cuts a small window around the reference position, applies a 3×3 median despiking and local-background subtraction, and computes a centre-of-mass centroid. A **second pass re-centres the window on the first-pass centroid**, which removes the inward bias that common tracking motion would otherwise introduce by clipping a spot’s wings — a correction that materially improves accuracy. (See [Appendix B.2](#))

15.3 Seeing Physics

For each doublet the per-frame separation $(x_1 - x_2, y_1 - y_2)$ is rotated into a **longitudinal** component (along the doublet vector) and a **transverse** component (perpendicular to it), and each is sigma-clipped to reject cloud/glitch frames. Their variances σ_l^2, σ_t^2 (converted to rad^2 via the plate scale) give the Fried parameter through the Sarazin & Roddier (1990) / Tokovinin (2002) relations, with D the sub-aperture diameter and d the hole separation:

$$\begin{aligned} \sigma_l^2 &= 2 \lambda^2 r_0^{-5/3} c_l, & c_l &= 0.179 D^{-1/3} - 0.0968 d^{-1/3} \\ \sigma_t^2 &= 2 \lambda^2 r_0^{-5/3} c_t, & c_t &= 0.179 D^{-1/3} - 0.145 d^{-1/3} \\ r_0 &= \left(\frac{2 \lambda^2 c}{\sigma^2} \right)^{3/5}, & \varepsilon_0 &= 0.98 \frac{\lambda}{r_0} \text{ (FWHM)}. \end{aligned}$$

The longitudinal and transverse estimates are computed separately; their agreement is a built-in sanity check (a large discrepancy flags wind shake or tracking problems).

15.4 Airmass Correction & Multi-Star Combination

- **To the Zenith:** Turbulence is measured along a slanted line of sight, so seeing worsens with airmass. The engine computes the target altitude from the header RA/Dec, the observation time, and the configured site (via `astropy AltAz`), forms the airmass X (Kasten & Young 1989), and corrects to the zenith: $\varepsilon_0 = \varepsilon X^{-0.6}$.

- **Combining Stars:** Every valid doublet yields an independent seeing estimate; these are combined with an SNR weighting for the final value, and the **star-to-star scatter** is reported as a quality metric. Using many doublets sharply improves the statistical precision and cadence over a single star.

15.5 Continuous Monitoring & Quality Control

A stdlib polling watcher scans the folder, deferring any file whose modification time is too recent (still being written) and catching up on a backlog quickly. Each frame is reduced *as it arrives* (see Wide-Field Operation below), and every *cadence* frames the monitor appends a timestamped record to the logs, refreshes the live plot, and rewrites `status_latest.json`. Each window is flagged with the reasons it passed or failed: too few frames or doublets, low valid-frame fraction, longitudinal/transverse disagreement, or a missing airmass. Bad windows are recorded but do not corrupt the trend.

15.6 Wide-Field Operation (0.5° Fields)

The 8-inch camera delivers a wide field (about half a degree, thousands of pixels across). Three features make this practical:

- **Stamp buffering (bounded memory):** Rather than buffering a window of full frames — which for a 0.5° field would be gigabytes — the monitor processes each frame on arrival. It keeps a single exponential-moving-average reference for detection and, per doublet, only a rolling deque of the differential offsets. Memory is therefore $\sim$ one frame regardless of the window length or field size; in tests, streaming 180 frames of 1500×1500 grew memory by ~ 0.1 GB where a full-frame buffer would have needed ~ 1.35 GB.
- **Ellipticity rejection:** A fast $f/5$ Newtonian has real coma off-axis. Each detected spot's ellipticity is measured from its second moments, and spots more elongated than a configurable limit are rejected, so aberrated field-edge stars do not degrade the result.
- **Central-field restriction:** Optionally, only stars within a configurable radius (in arcminutes) of the field centre are used — a simple way to confine the measurement to the best-corrected part of the field while still averaging many doublets.

Because many stars across the field each yield an independent seeing estimate, a wide field is a statistical advantage: the values are combined for precision, and their scatter is reported as a quality metric.

16 Appendix: DIMM Technical Details

16.1 Differential Image Motion & the Fried Parameter

The Fried parameter r_0 is the diameter over which the atmospheric wavefront stays roughly coherent; larger r_0 means better seeing. A single aperture's image position wanders with the wavefront tilt averaged over that aperture, but this includes telescope tracking motion that we cannot separate from the atmosphere.

- **Why Differential:** Two sub-apertures on the same telescope see the same tracking motion but statistically independent atmospheric tilts. Subtracting their centroids cancels the common motion exactly, leaving a differential variance that depends only on the turbulence and the fixed geometry (D, d) — which is why a DIMM needs no absolute pointing stability, only that both spots stay on the detector.

- **Two Axes:** The longitudinal (baseline-parallel) and transverse variances have different coefficients because turbulence correlates differently along and across the sub-aperture baseline; measuring both provides two independent seeing values and a consistency check.

16.2 Two-Pass Centroiding Against Tracking Motion

A fixed measurement window centred on a spot's mean position is dangerous: when common tracking motion carries the spot toward the edge of its window, the far wing of the point-spread function is clipped and the centre-of-mass is pulled *inward*. Because both spots suffer this pull, the measured differential motion is damped and the seeing is biased artificially good.

- **The Fix:** The engine centroids once to locate the spot, then re-extracts the window centred on that first estimate and centroids again. The spot is now centred in its own window regardless of where common motion carried it, so the full PSF is captured and the differential motion — and hence the seeing — is recovered without bias.